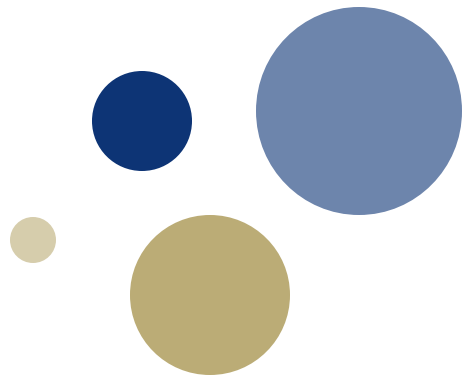




NTNU

Kunnskap for en bedre verden

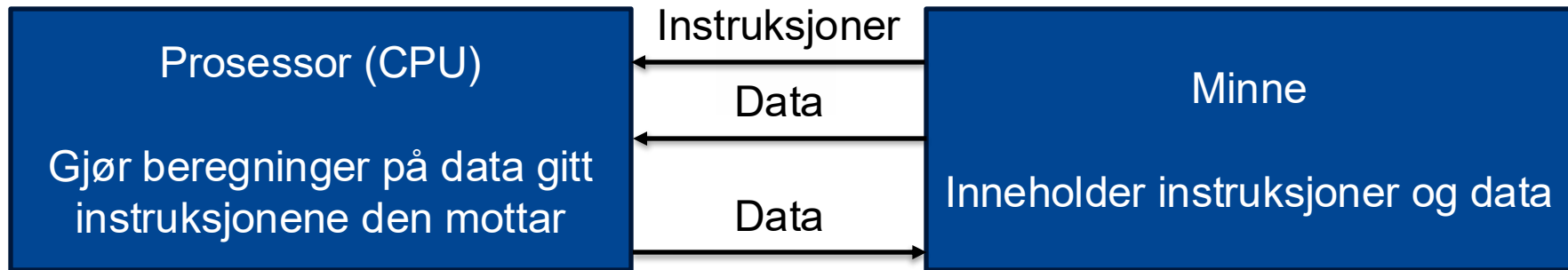


Forelesning 4: Enkeltsykelprosessor

TDT4160 Datamaskiner

Prinsippet om lagrede program

Dette kan vi nå



Men hva er det som skjer inni her?

Hva er en prosessor?

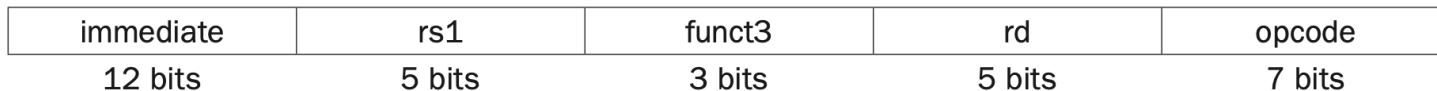
Prosesor = en maskin som utfører instruksjoner

R-type



F. eks. *add*

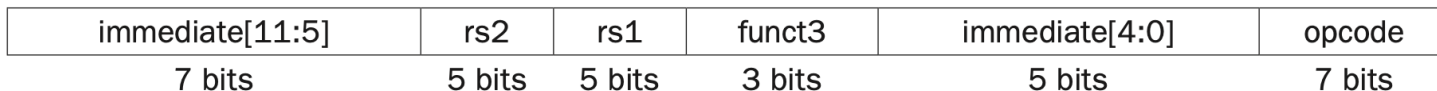
I-type



F. eks. *addi* og *lw*

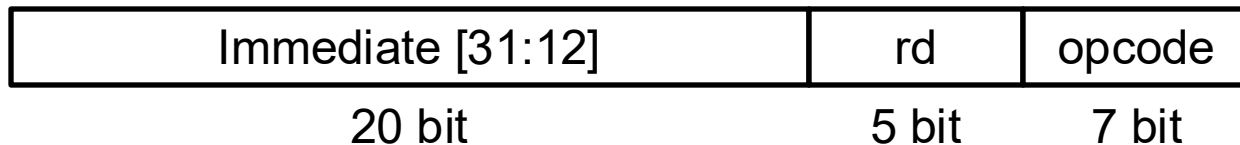
S-type

F. eks. *sw*



U-type

F. eks. *lui*



Ytelse

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$


Prosessoren må utføre de instruksjonene som er i programmet

... men prosessoren kan gjøre det effektivt gjennom å bruke færre klokkesykler per instruksjon og færre sekunder per klokkesykel



Tema 3.1: Enkeltsykelprosessor (Kapittel 4.1 og 4.2)

DATASTI OG KONTROLLENHET

Læringsutbytte T3.1

- Studenten skal kunne forklare begrepene **datasti** og **kontrollenhet**.
- Studenten skal kunne forklare mikroarkitekturen til **enkeltsykelprosessoren** (kunne gjengi og forklare figur 4.21).
- Studenten skal kunne sette opp rett **kontrollord** for en instruksjon, inkludert korrekt bruk av «don't care».
- Studenten skal kunne **identifisere instruksjonstype** fra et kontrollord.

Læringsutbytte T3.1

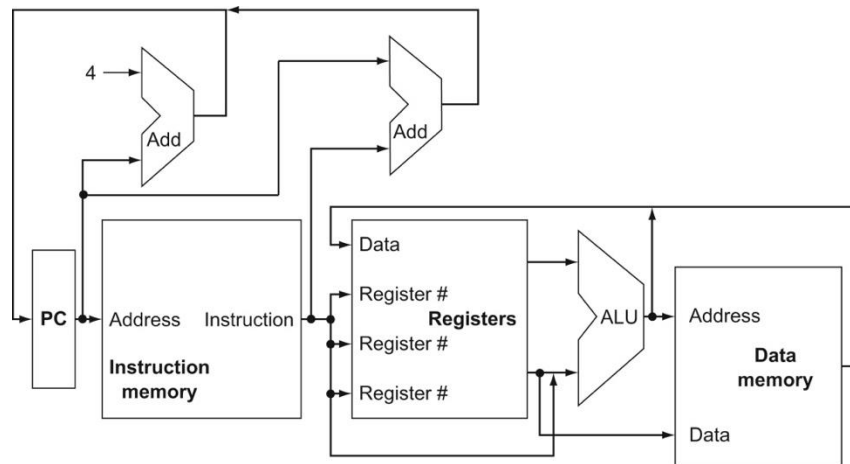
- Studenten skal kunne forklare begrepene **datasti** og **kontrollenhet**.
- Studenten skal kunne forklare mikroarkitekturen til enkeltsykelprosessoren (kunne gjengi og forklare figur 4.21).
- Studenten skal kunne sette opp rett kontrollord for en instruksjon, inkludert korrekt bruk av «don't care».
- Studenten skal kunne identifisere instruksjonstype fra et kontrollord.

Forenkling

- Vi ser på en mikroarkitektur som støtter følgende instruksjoner:
 - Minne: Load word (lw) og store word (sw)
 - Aritmetikk: `add`, `sub`, `and`, **og** `or`
 - Forgreining: `beq`
- Dette betyr at vi kan se bort fra U-type instruksjoner og må støtte relativt få operasjoner
 - ... men å legge til flere instruksjoner gir ikke noe fundamentalt nytt, det er bare mer jobb

Hva må prosessoren kunne gjøre?

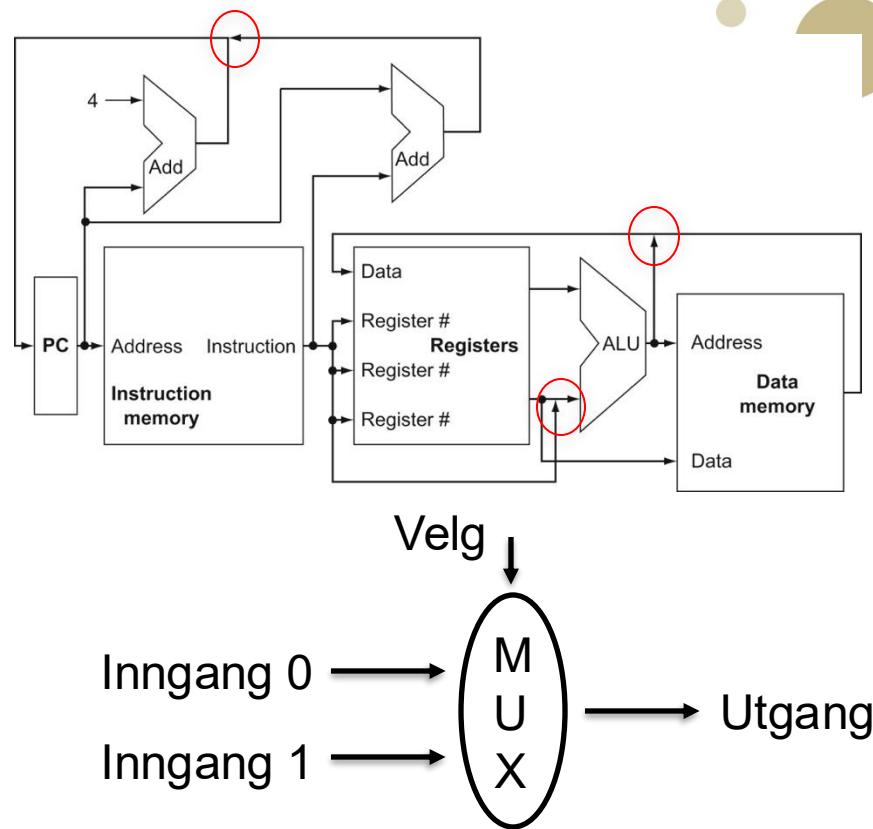
- Alle instruksjonene våre må:
 - Hente instruksjonen i minnet basert på adressen til programtelleren («Program Counter (PC)»)
 - Lese minst ett register
- Vi må også kunne:
 - Skrive maksimalt et register
 - Lese og skrive til minnet
 - Endre programtelleren (forgreining)
- Boksene er enheter (delsystemer)
 - ... altså transistorer og ledninger
- Strekene er ledninger
 - ... og verdien på en eller flere ledninger kalles ofte et signal



Abstraksjon!

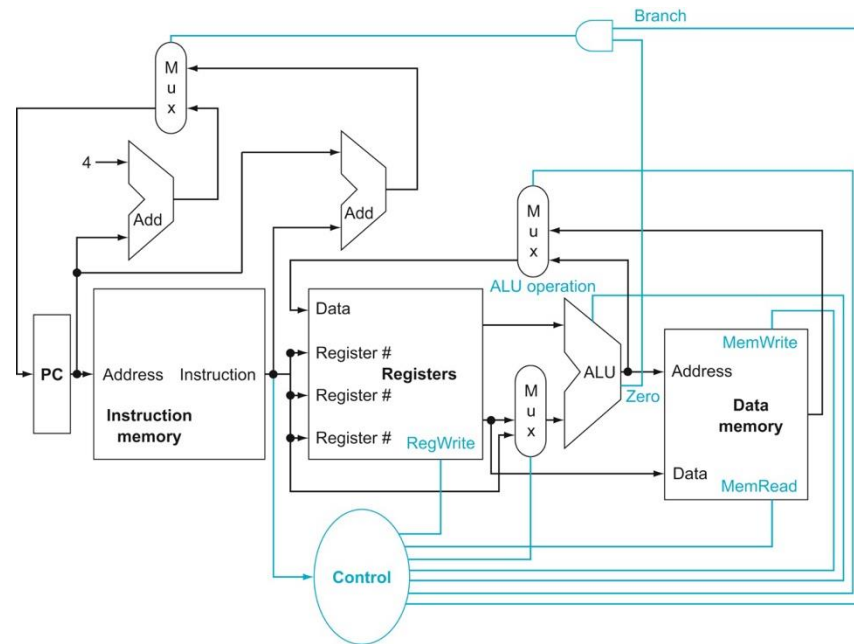
Multiplekser

- Vi kan ikke bare koble ledninger sammen
 - Hvis to enheter prøver å drive en ledning til ulike verdier er det uvisst hvilken verdi vi får ut
- Løsning: Multiplekser
 - Hvis velg er 0, legges verdien på inngang 0 på utgangen
 - Hvis velg er 1, legges verdien på inngang 1 på utgangen



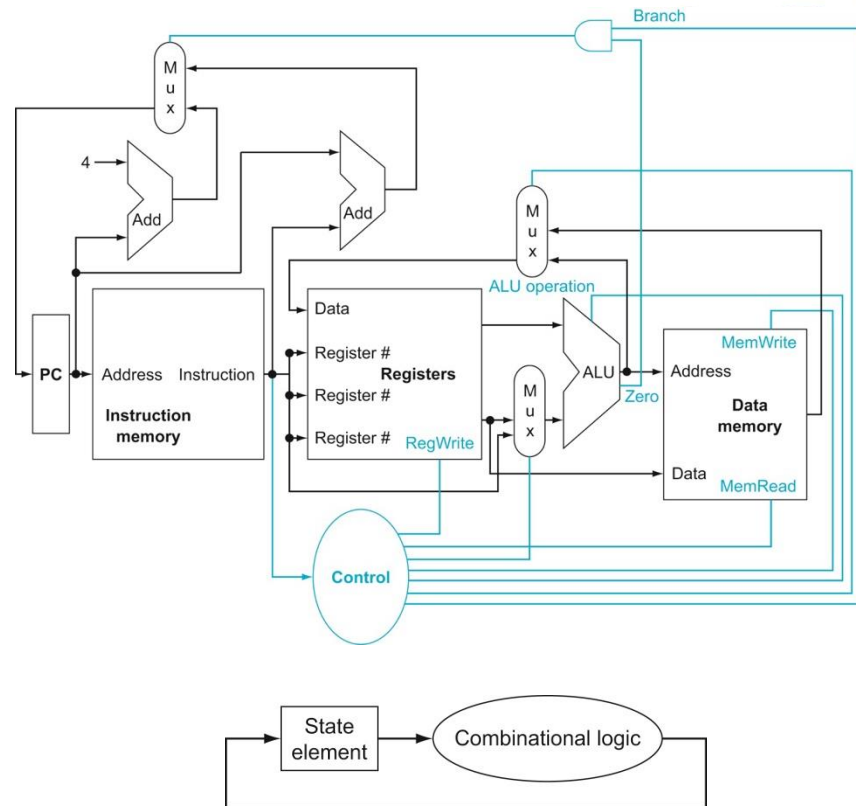
Datasti og kontrollenhet

- Vi lager en datasti («datapath») som kan utføre alle instruksjonene
- Vi bestemmer hvilke enheter som skal være aktive med kontrollsignaler
 - ... som enten går til multipleksere eller direkte til en enhet
- Kontrollenheten setter kontrollsignalene basert på operasjonskoden(e) i instruksjonen



Kombinatorisk og sekvensiell logikk

- Datamaskiner er digitale og synkrone
 - Verdiene er enten 0 eller 1 og oppførselen styres av et klokkesignal
- Det finnes to typer digital logikk:
 - *Kombinatorisk logikk* husker ingenting er uavhengig av klokkesignalet
 - *Sekvensiell logikk* husker ting (har minne) og er typisk avhengig av klokkesignalet
- En enkeltskykelprosessor gjør alt arbeidet i en klokkesykel
 - ... og vi trenger derfor nesten bare kombinatorisk logikk (unntak: registerfila og minnene)
- Ekstraforelesning 2 dekker det dere må kunne om sekvensiell logikk
 - ... altså låser, vipper, register, registerfil, og tilstandsmaskin





Tema 3.1: Enkeltsykelprosessor (Kapittel 4.3 og 4.4)

VI LAGER EN ENKELTSYKELPROSESSOR

Læringsutbytte T3.1

- Studenten skal kunne forklare begrepene datasti og kontrollenhet.
- Studenten skal kunne forklare mikroarkitekturen til **enkeltsykelprosessoren** (kunne gjengi og forklare figur 4.21).
- Studenten skal kunne sette opp rett kontrollord for en instruksjon, inkludert korrekt bruk av «don't care».
- Studenten skal kunne identifisere instruksjonstype fra et kontrollord.

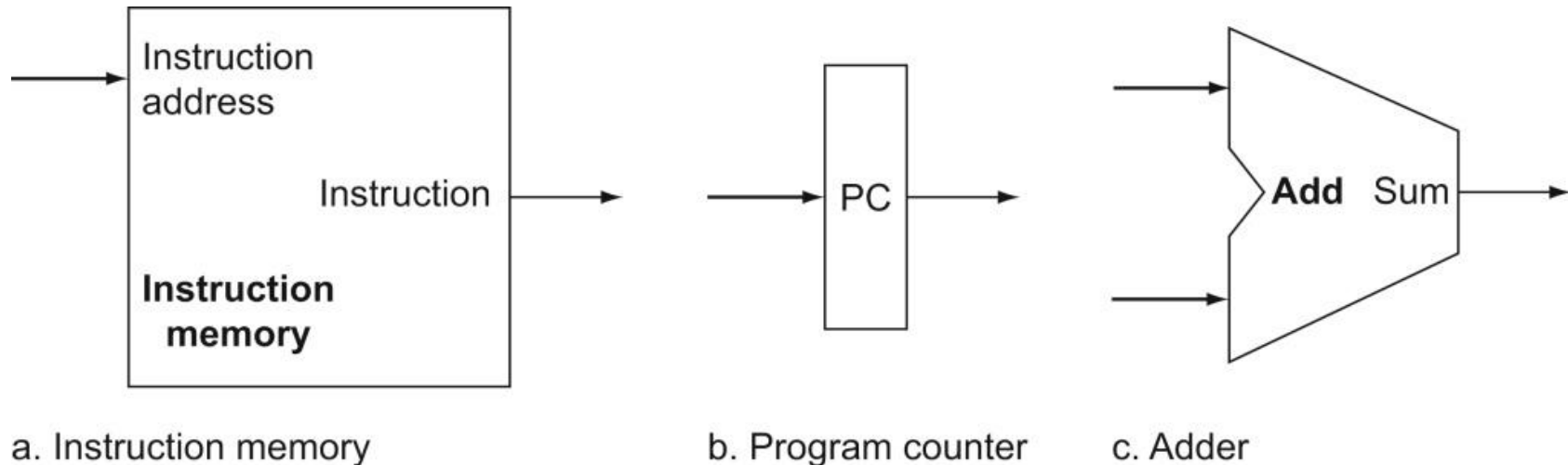
Nå skal vi lage vår første prosessor

- Enkeltskykelprosessor = en prosessor som utfører alle instruksjoner på én klokkesykel
- Da må vi i prinsippet håndtere alle instruksjonsformat:

Name (Field size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

.. men vi skal «bare» støtte `lw` , `sw` , `add` , `sub` , `and` , `or` og `beq`

Hva trenger alle instruksjoner?

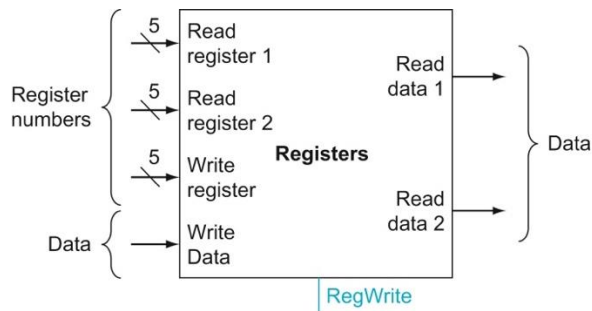


Hvordan skal disse kobles sammen for å hente neste instruksjon?

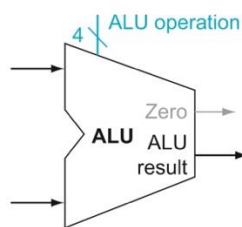
Hvordan hente instruksjoner?



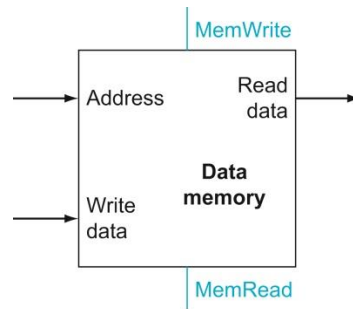
R-type og I-type instruksjoner trenger også:



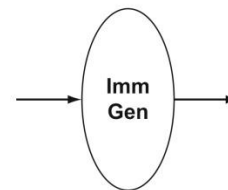
a. Registers



b. ALU



a. Data memory unit



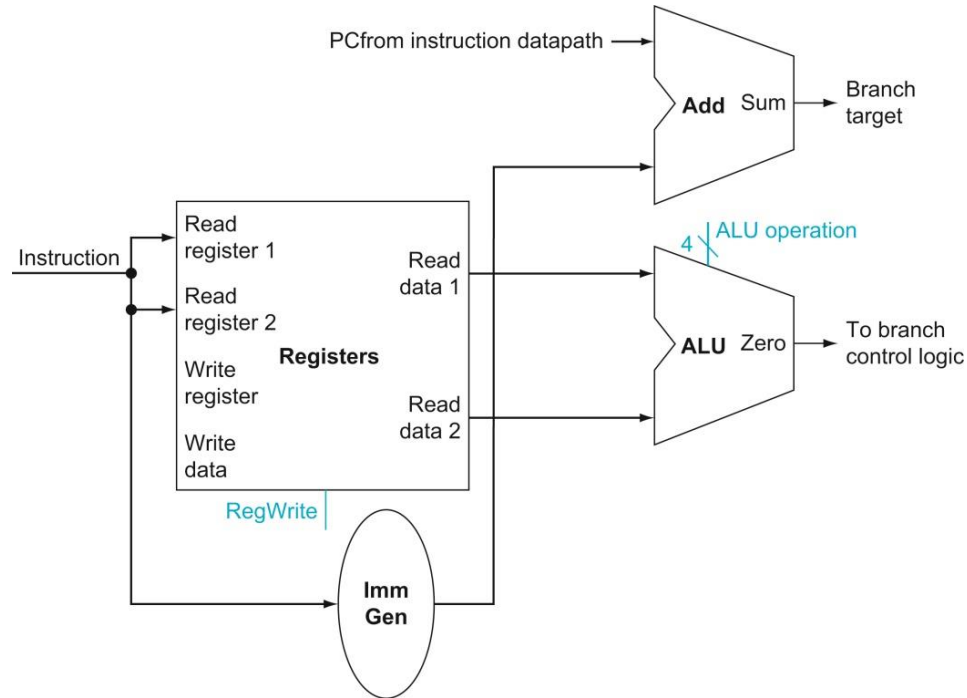
b. Immediate generation unit

Hvordan skal disse kobler vi disse sammen med det vi allerede har?

Vi lager en prosessor for R- og I-type instruksjoner



Forgreningsinstruksjoner

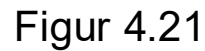


Hvordan skal disse kobler vi dette sammen med det vi allerede har?

**Vi lager en prosessor som også
støtter beq-instruksjonen**



22





Tema 3.1 (Kapittel 4.3 og 4.4)

KONTROLLORD

Læringsutbytte T3.1

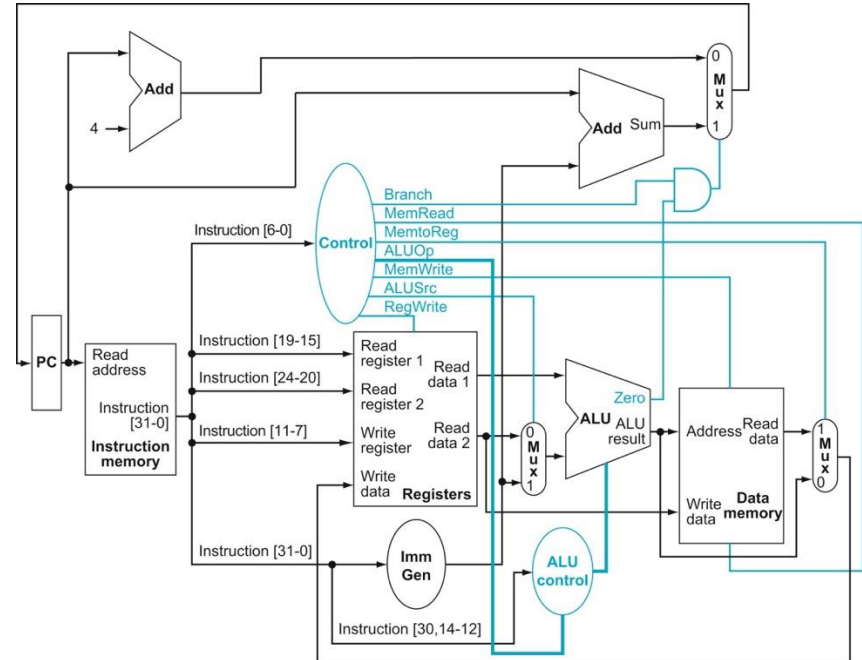
- Studenten skal kunne forklare begrepene datasti og kontrollenhet.
- Studenten skal kunne forklare mikroarkitekturen til enkeltsykelprosessoren (kunne gjengi og forklare figur 4.21).
- Studenten skal kunne sette opp rett **kontrollord** for en instruksjon, inkludert korrekt bruk av «don't care».
- Studenten skal kunne **identifisere instruksjonstype** fra et kontrollord.

Kontrollord

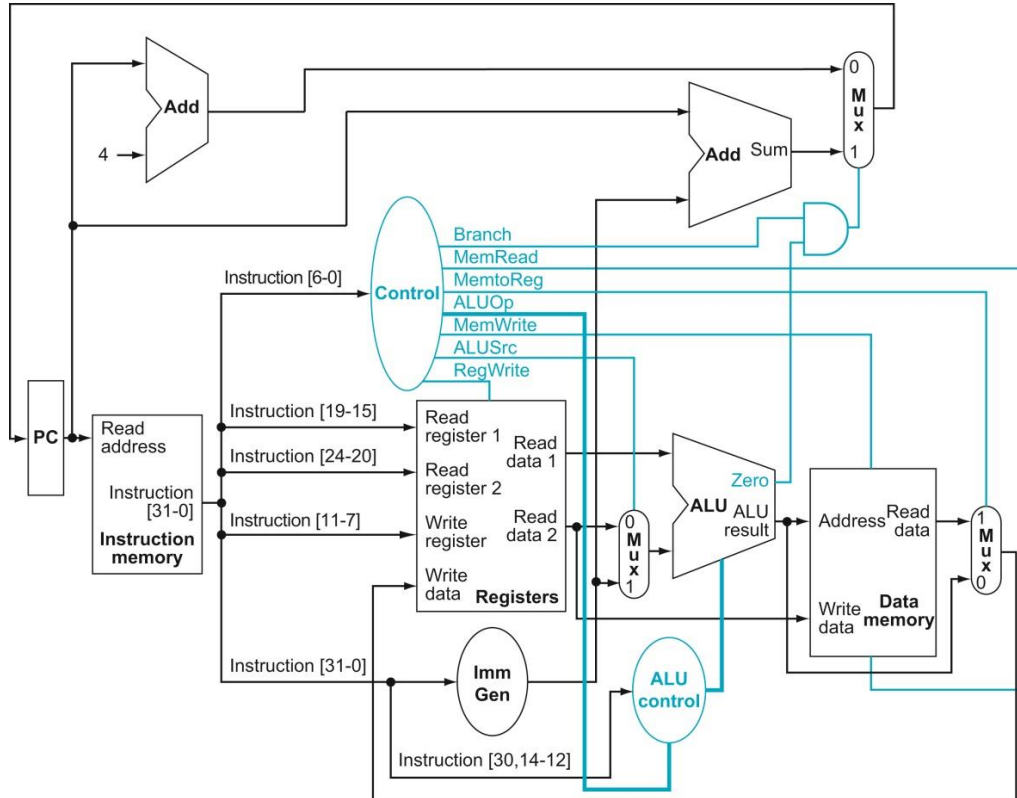
- Kontrollordet er signalene som går ut fra kontrollenheten for en spesifikk instruksjon
- Hvordan får vi prosessoren til å utføre:
 - Add
 - Lw
 - Sw
 - Beq

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXXX	XXX	subtract	0110
R-type	10	add	00000000	000	add	0010
R-type	10	sub	01000000	000	subtract	0110
R-type	10	and	00000000	111	AND	0000
R-type	10	or	00000000	110	OR	0001



Eksempel: add x3, x1, x2



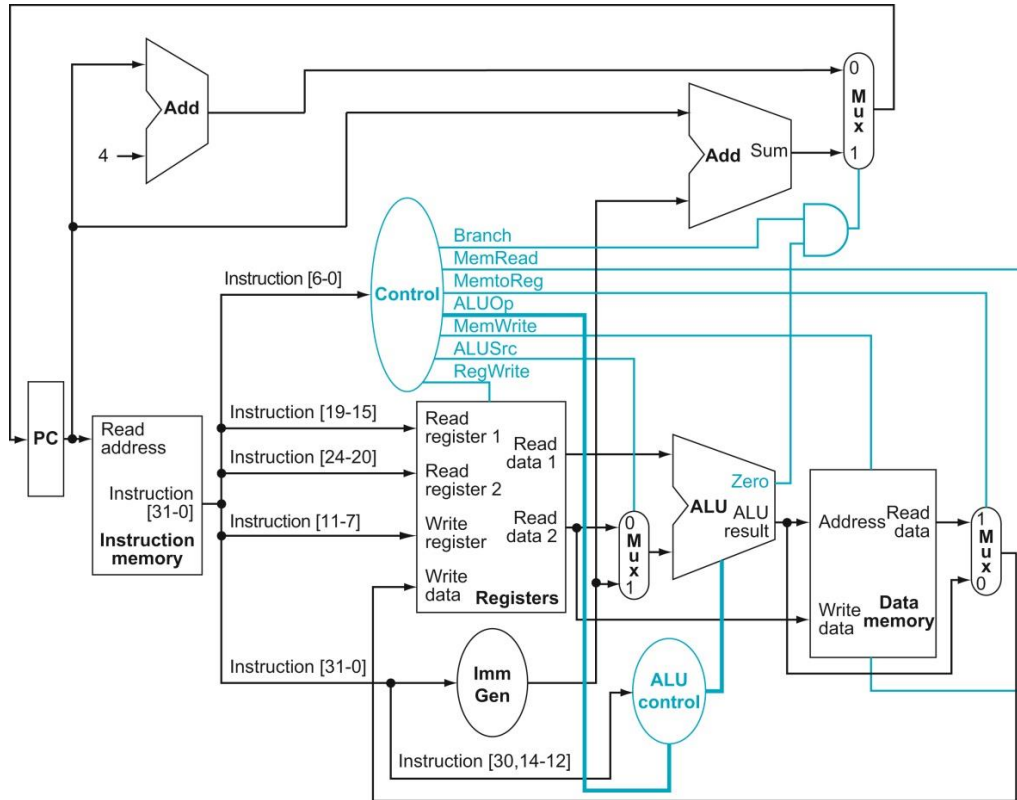
ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXX	XXX	add	0010
sw	00	store word	XXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Register	Verdi
x1	10
x2	20
x3	12

Quiz!

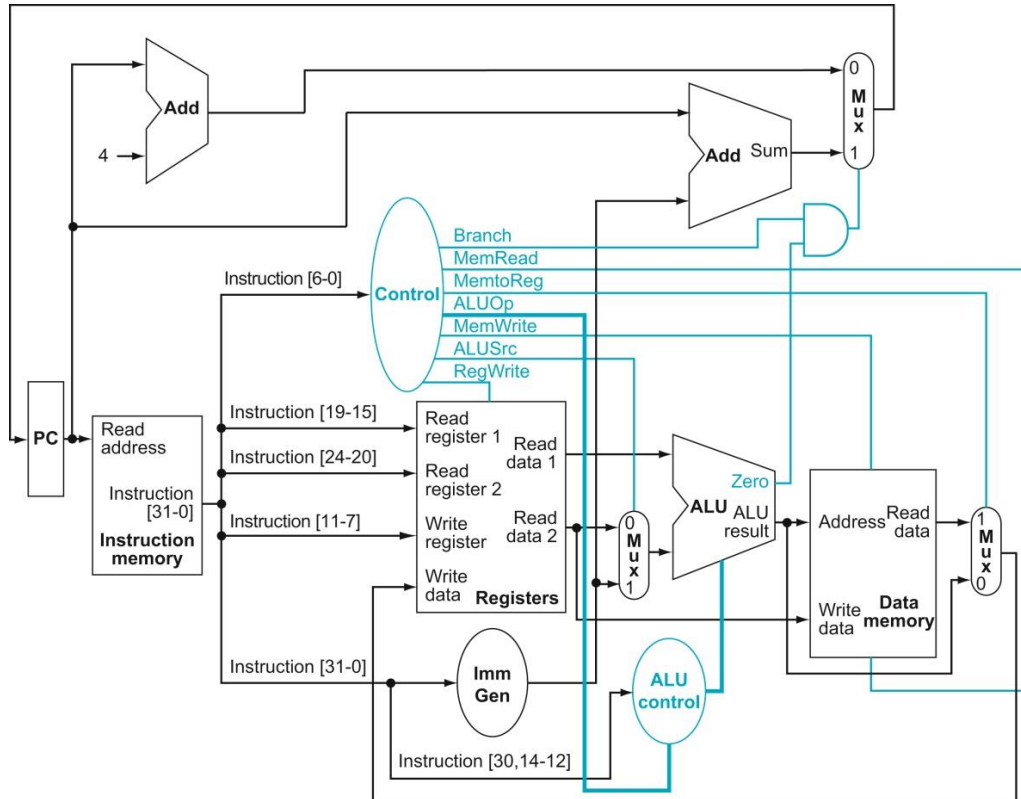
Eksempel: lw x5, 8(x4)



Register	Verdi
x4	1024
x5	20

Minneadresse	Verdi
1032	17
1028	16
1024	15

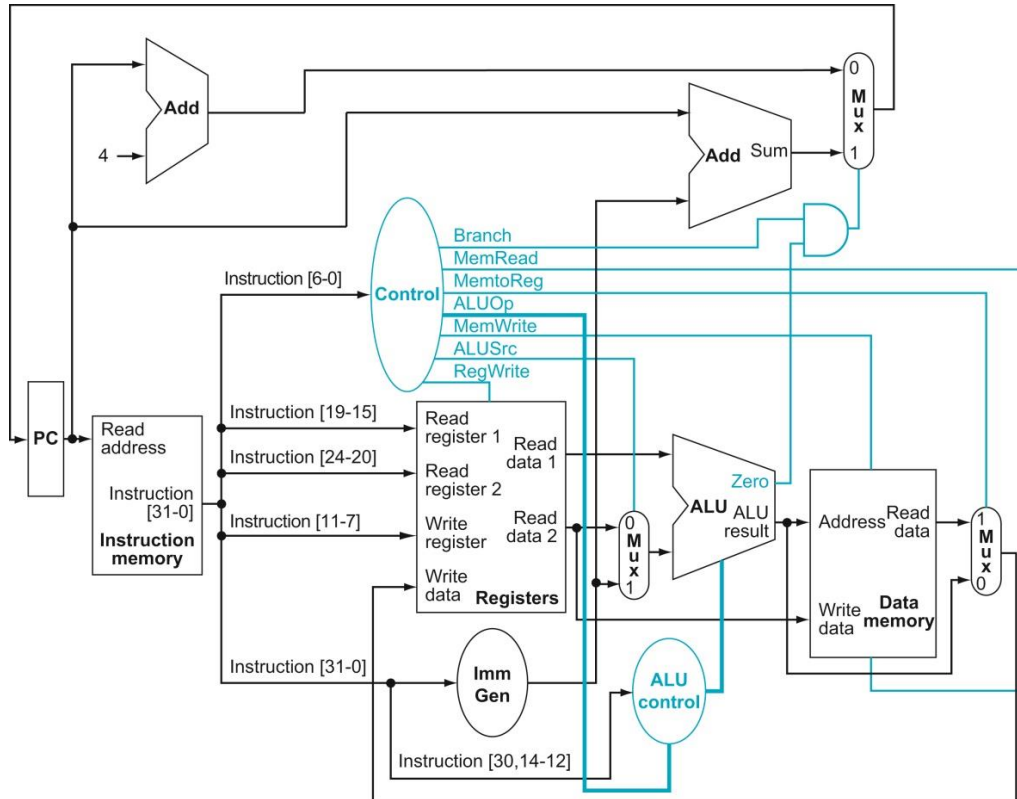
Eksempel: `sw x10, -4(x1)`



Register	Verdi
x1	1032
x10	2

Minneadresse	Verdi
1032	17
1028	16
1024	15

Eksempel: beq x1, x2, 8

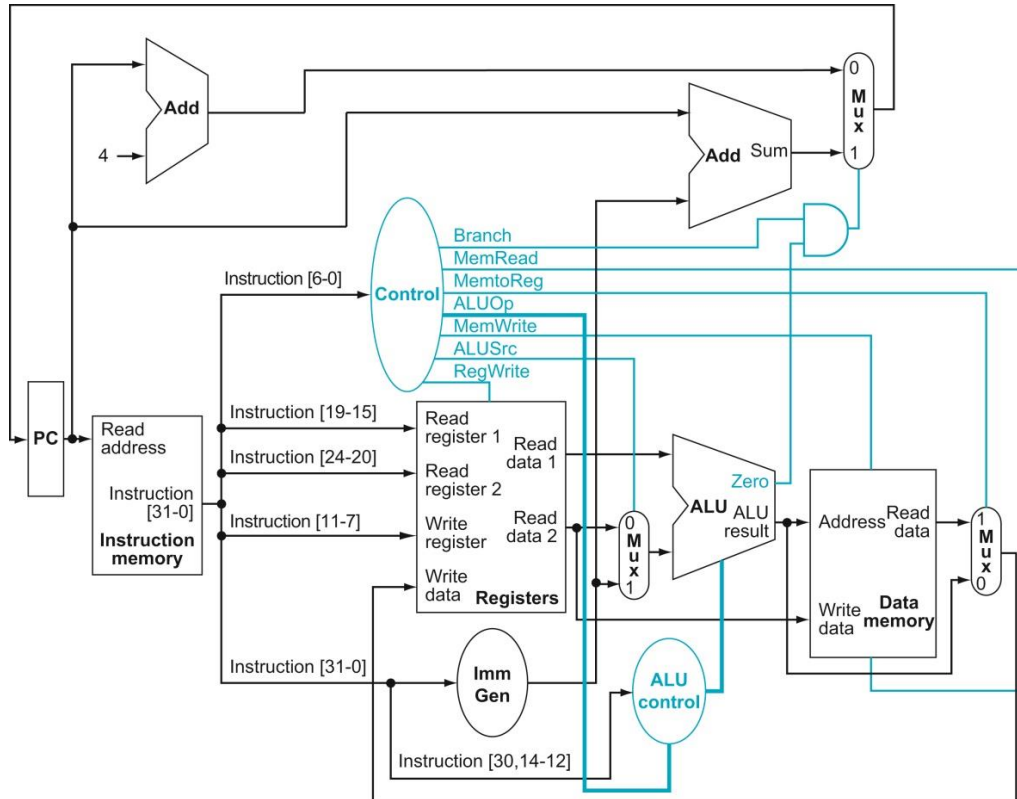


PC = 16

Register	Verdi
x1	2
x2	2

Minneadresse	Verdi
24	sub [...]
20	add [...]
16	beq x1, x2, 8

Eksempel: beq x1, x2, 8



PC = 16

Register	Verdi
x1	15
x2	2

Minneadresse	Verdi
24	sub [...]
20	add [...]
16	beq x1, x2, 8



Tema 3.1 (Kapittel 4.3 og 4.4)

KONTROLLENHETENE

Læringsutbytte T3.1

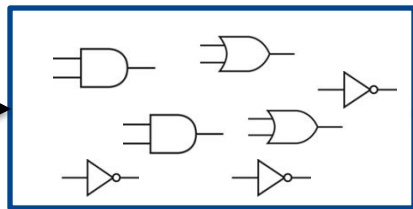
- Studenten skal kunne forklare begrepene datasti og kontrollenhet.
- Studenten skal kunne forklare mikroarkitekturen til enkeltsykelprosessen (kunne gjengi og forklare figur 4.21).
- Studenten skal kunne sette opp rett **kontrollord** for en instruksjon, inkludert korrekt bruk av «don't care».
- Studenten skal kunne **identifisere instruksjonstype** fra et kontrollord.

Hvordan lager vi kontrollenheter?

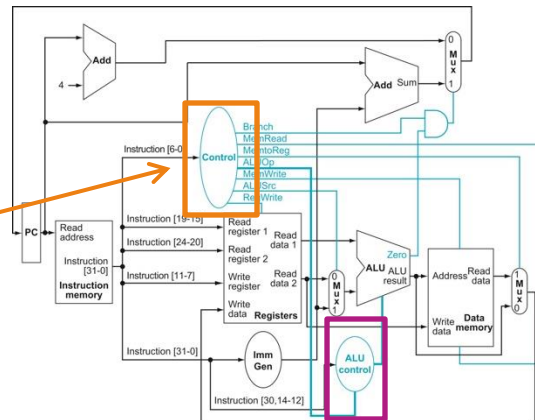
	Input	Output						
	<i>Opcode</i>	<i>Branch</i>	<i>MemRead</i>	<i>MemtoReg</i>	<i>ALUOp</i>	<i>MemWrite</i>	<i>ALUSrc</i>	<i>RegWrite</i>
add	0110011	0	0	0	10	0	0	1
lw	0000011	0	1	1	00	0	1	1
	...							

En (gigantisk) sannhetstabell som kan implementeres med kombinatorisk logikk

Opcode

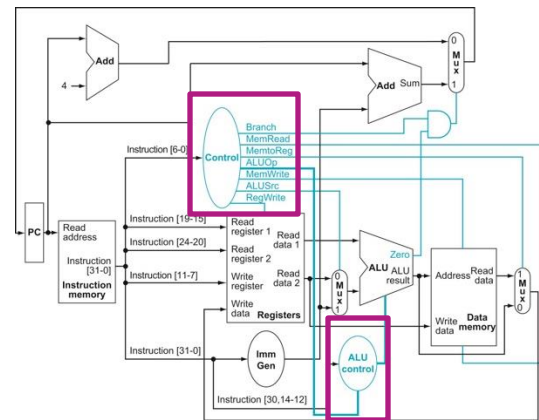


Kontrollord

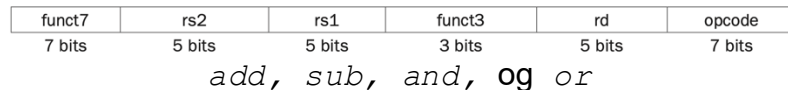


Enkeltskykelprosessoren har to kontrollenheter

- Grunnen er at å dele kontrollenheten i to gjør sannhetstabellene mindre:
 - Sannhetstabeller har inntil 2^i linjer (der i er antall bit i input)
 - «Control» tar kun opcode (7 bit) som input ($2^7 = 128$)
 - «ALU Control» tar ALUOp, func7, og func3 som input ($2^{2+7+3} = 2^{12} = 4096$)
 - En kombinert kontrollenhet gir $2^{12+6} = 2^{18} = 262144$ linjer, altså betydelig mer enn $4096+128$
- Vi skal også kun støtte lw, sw, beq, add, sub, and, og or.
 - ... og da blir det mange muligheter for bruk av «don't care» som reduserer antallet linjer betydelig



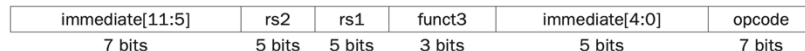
R-type



I-type



S-type



SW

Eksempel: Sannhets- tabell for «ALU Control»

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Kontrollsignaler til ALU

Instruksjon	Innverdier			Utverdier	Ønsket ALU operasjon
	<i>ALUOP</i>	<i>Func7 (kun R-type)</i>	<i>Func3</i>		
lw					Addisjon
sw					Addisjon
beq					Subtraksjon
add					Addisjon
sub					Subtraksjon
and					Logisk og
or					Logisk eller

Sannhetstabell for «Control»

- Opcode bruker bit [6:0] i alle instruksjonsformat
- Sannhetstabellen for «Control» tilordner dermed kun opcode til kontrollsignalene rett kontrollsignal i datastien
- Merk at denne (og forrige) sannhetstabell kun viser de innverdiene prosessoren vår støtter
 - ... mens en ekte implementasjon må ha definerte utgangsverdier for alle mulige inngangsverdier

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1



Tema 3.1 (Kapittel 4.3 og 4.4)

OPPSUMMERING

Så enkelt kan man lage en datamaskin

- Vi lager en **datasti** som kan gjøre alle de tingene som alle instruksjoner trenger å gjøre
- Vi lager en **kontrollenhet** som skrur på nøyaktig de enhetene som den instruksjonen vi utfører akkurat nå trenger
- Neste skritt:
 - Vi har enda ikke beskrevet ALU, registerfila og minnene ordentlig
 - Ytelsen til denne datamaskinen er omtrent så dårlig som den kan bli

